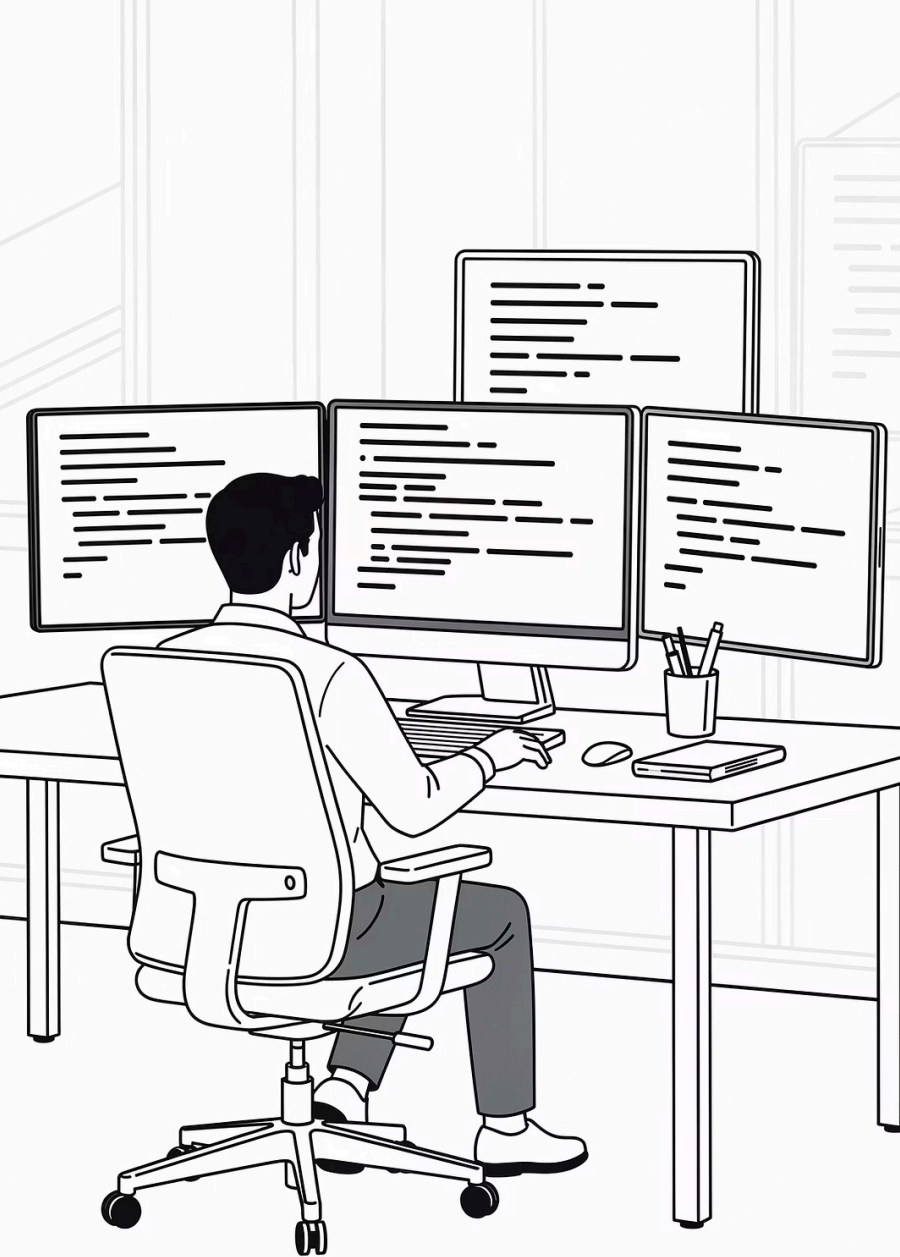




Spring – Bonnes Pratiques & Architecture

Synthèse des pratiques essentielles pour construire des applications Spring robustes, maintenables et prêtes pour la production.

Ce que vous avez déjà acquis

Java EE

Servlets, JSP, architecture entreprise

Spring & Spring Boot

Écosystème, Initializr, auto-configuration

Configuration & DI

IoC Container, @Component, @Autowired

Spring AOP



Synthèse & Bonnes Pratiques Spring

Ce module consolide vos acquis et les met en perspective : configuration, patterns d'architecture, et préparation à la persistance des données.

CHAPITRE 2

Configuration – Bonnes Pratiques

À privilégier

- Classes `@Configuration` dédiées
- `@ConfigurationProperties` typées
- Profils par environnement (`@Profile`)
- Secrets via variables d'environnement

À éviter

- `@Value` dispersés dans tous les beans
- Configuration de production en dur dans le code
- Mots de passe ou clés API versionnés

 Ajoutez `@Validated` sur vos `@ConfigurationProperties` pour valider les propriétés au démarrage.

@ConfigurationProperties en pratique

La solution recommandée pour gérer la configuration dans un projet Spring Boot professionnel — bien plus maintenable que de multiplier les @Value.

```
@Configuration
@ConfigurationProperties(prefix = "app")
@Validated
public class AppProperties {

    @Valid
    private Security security = new Security();

    @Data
    public static class Security {
        @NotBlank
        private String jwtSecret;
        @Positive
        private long tokenExpirationMs;
    }
}
```

Organisation des packages

Par couche (*traditionnel*)

```
src/main/java/  
├── controller/  
│   ├── UserController  
│   └── OrderController  
├── service/  
├── repository/  
└── model/
```

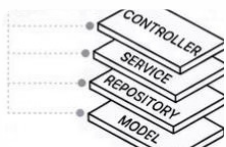
- ✓ Simple à comprendre
- ❑ Couplage fort entre domaines

Par domaine (*DDD*)

```
src/main/java/  
├── user/  
│   ├── UserController  
│   ├── UserService  
│   └── UserRepository  
├── order/  
└── shared/  
    ├── config/  
    └── exception/
```

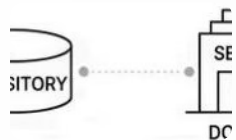
- ✓ Haute cohésion, faible couplage
- ✓ Facilite la transition microservices

Patterns d'architecture Spring



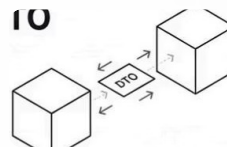
Architecture en couches

Séparation des responsabilités entre Controller, Service, Repository et Model.



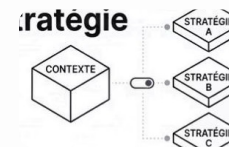
Pattern Repository

Abstraction de l'accès aux données, découplant la logique métier de la persistance.



Pattern DTO

Transfert de données entre couches, distinct des entités JPA.



Pattern Stratégie

Comportements interchangeables via des interfaces — idéal avec l'injection de dépendances Spring.

Le Service Layer – Cœur de l'application

Ce qu'il fait

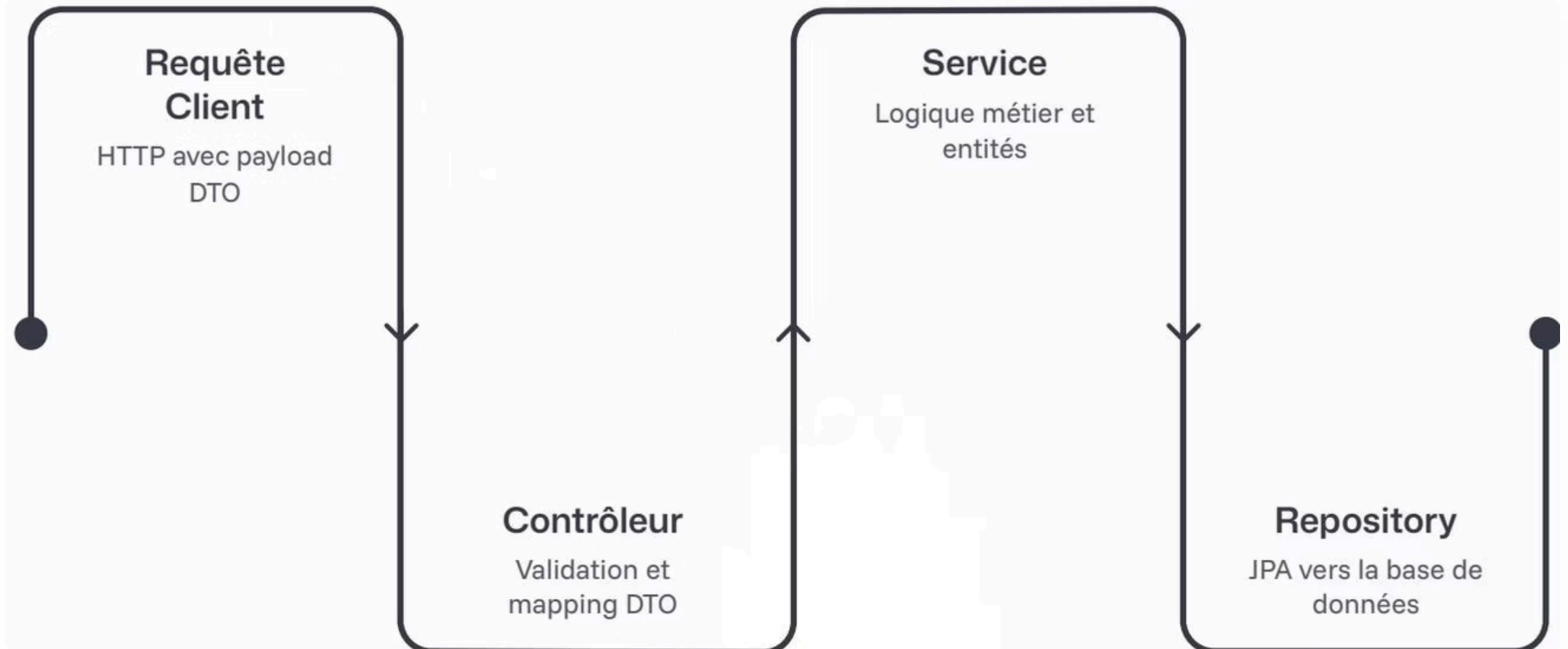
- Orchestrer les appels aux repositories
- Appliquer les règles métier
- Gérer les transactions (@Transactional)
- Mapper entités → DTOs
- Lever des exceptions métier explicites

Ce qu'il ne fait PAS

- Accéder directement à la base de données
- Construire des réponses HTTP
- Connaître le protocole de transport
- Valider le format des entrées (rôle du controller)

✔ Un Service Layer bien défini rend l'application testable unitairement sans démarrer Spring ni avoir de base de données.

Le Pattern DTO



Sans DTO, l'entité JPA est directement exposée dans l'API REST — toute modification du schéma BDD impacte immédiatement le contrat d'API. Les DTOs constituent une **couche anti-corruption** qui isole ces deux préoccupations.

DTOs avec MapStruct

MapStruct automatise le mapping entité ↔ DTO à la compilation, sans overhead à l'exécution.

```
// DTO de réponse — passwordHash exclu
public record UserResponseDto(
    Long id, String firstName,
    String lastName, String email,
    LocalDateTime createdAt) {}

// Mapper généré automatiquement
@Mapper(componentModel = "spring")
public interface UserMapper {
    UserResponseDto toDto(User user);

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "passwordHash", ignore = true)
    User toEntity(CreateUserDto dto);
}
```

Gestion centralisée des exceptions

Plutôt que de multiplier les `try-catch`, Spring MVC offre `@RestControllerAdvice` pour traiter toutes les exceptions en un seul endroit.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiError> handleNotFound(
        ResourceNotFoundException ex,
        HttpServletRequest request) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ApiError(404, ex.getMessage(),
                request.getRequestURI(), Instant.now()));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiError> handleAll(Exception ex) {
        log.error("Exception non gérée", ex);
        return ResponseEntity.status(500)
            .body(ApiError.internalError());
    }
}
```

Hiérarchie d'exceptions métier



Principe

Chaque exception métier porte un code d'erreur unique et un message localisable. Le `GlobalExceptionHandler` mappe chaque type vers le code HTTP sémantiquement correct.

- **NotFoundException** → 404
- **BusinessException** → 422 (règle métier violée)
- **ApplicationException** → base applicative
- **RuntimeException** → fondation Java

Validation avec Bean Validation

@NotNull / @NotBlank

Valeurs obligatoires. @NotBlank interdit aussi les chaînes vides ou composées d'espaces.

@Size / @Min / @Max

Contraintes de taille pour les chaînes, collections, et de valeur pour les nombres.

@Email / @Pattern

Validation de format : email valide ou expression régulière personnalisée.

Contraintes personnalisées

Créez vos propres annotations avec @Constraint pour des règles métier complexes (unicité, format spécifique).

Events Spring – Communication découplée

Plutôt qu'un service A appelle directement un service B, il publie un événement. Ce pattern favorise le principe Open/Closed et facilite l'ajout de fonctionnalités sans modifier les composants existants.

```
// Publier un événement
eventPublisher.publishEvent(
    new UserRegisteredEvent(user.getId(), user.getEmail()));

// Écouter dans un autre composant
@Component
public class WelcomeEmailListener {
    @EventListener
    @Async
    public void onUserRegistered(UserRegisteredEvent event) {
        mailService.sendWelcomeEmail(event.getEmail());
    }
}

// Après commit de la transaction
@TransactionalEventListener(phase = AFTER_COMMIT)
public void onUserRegistered(UserRegisteredEvent event) {
    auditService.log("USER_REGISTERED", event.getId());
}
```

Gestion du cache avec Spring Cache



@Cacheable

Met en cache le résultat au premier appel. Les appels suivants retournent la valeur en cache. Idéal pour les données de référence :

```
@Cacheable("users")
```



@CacheEvict

Invalide une entrée ou vide le cache lors d'une mise à jour :

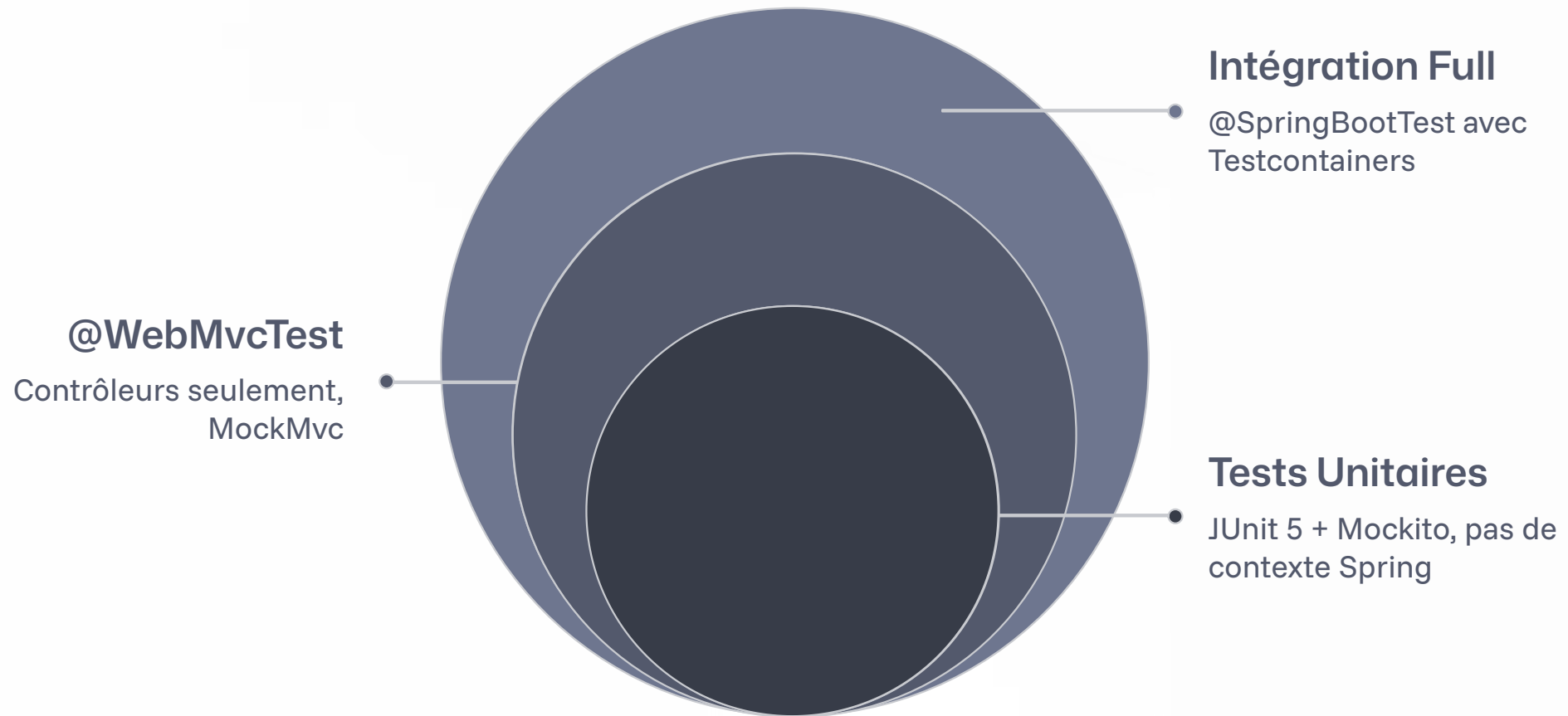
```
@CacheEvict(value="users",  
key="#userId")
```



@CachePut

Met à jour le cache sans court-circuiter l'exécution. La méthode s'exécute toujours — utile après une opération d'écriture.

Patterns de test Spring



Chaque niveau offre un compromis entre vitesse d'exécution et couverture. Privilégiez les tests unitaires pour la logique métier, et réservez @SpringBootTest aux tests d'intégration critiques.

Spring Boot Actuator – Observabilité

/actuator/health

État de santé de l'application et ses dépendances. Intégré dans les probes Kubernetes liveness/readiness.

/actuator/metrics

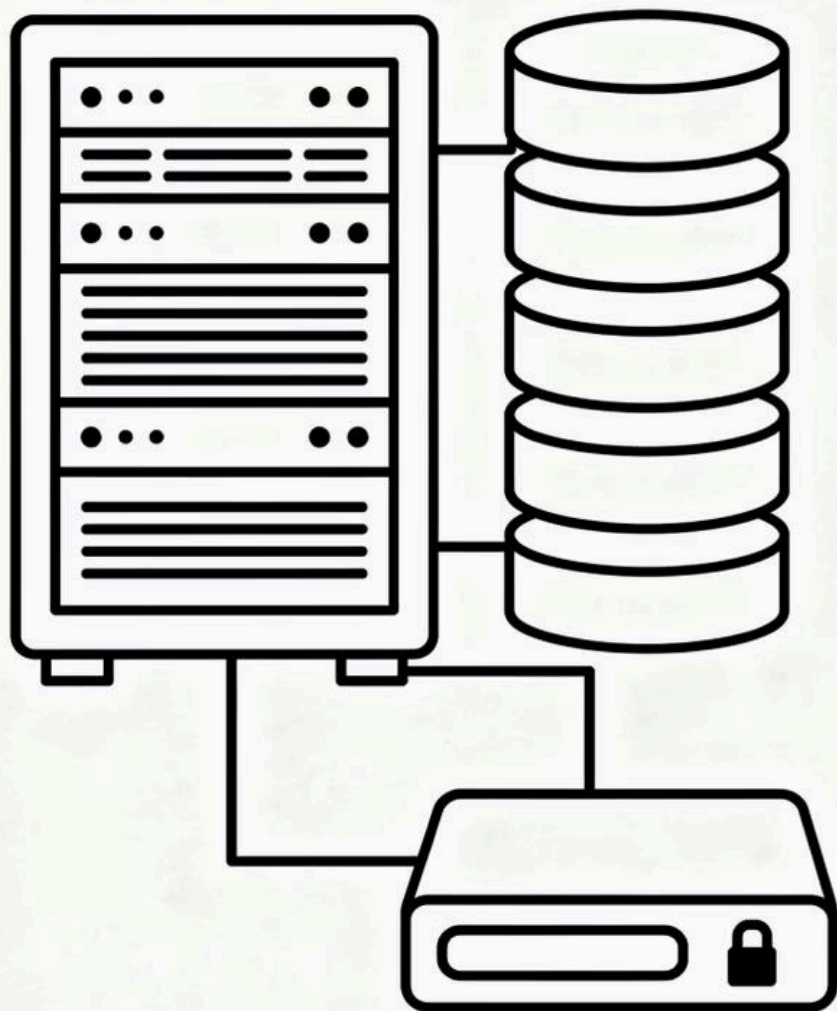
Métriques JVM, HTTP, base de données. Compatible Prometheus + Grafana.

/actuator/info

Version, commit Git, date de build — généré automatiquement via le plugin Maven/Gradle.

/actuator/loggers

Modifier dynamiquement le niveau de log d'un package **sans redémarrage**. Précieux pour le diagnostic en production.



Bonnes Pratiques de Sécurité

→ Mots de passe

Toujours `BCryptPasswordEncoder` ou `Argon2` — jamais de stockage en clair.

→ HTTPS en production

Certificat TLS obligatoire. Spring Boot supporte la configuration SSL dans `application.properties`.

→ Validation des entrées

Bean Validation sur tous les DTOs, requêtes paramétrées JPA — jamais de concaténation SQL.

→ Moindre privilège

Un utilisateur BDD dédié par application avec uniquement `SELECT/INSERT/UPDATE/DELETE`, jamais `DROP`.

Checklist de mise en production



Sécurité

- HTTPS activé, HTTP redirigé
- Actuator sécurisé
- Secrets dans variables d'environnement



Performance

- Pool HikariCP configuré
- Cache activé pour les données chaudes
- Index BDD sur colonnes de recherche



Observabilité

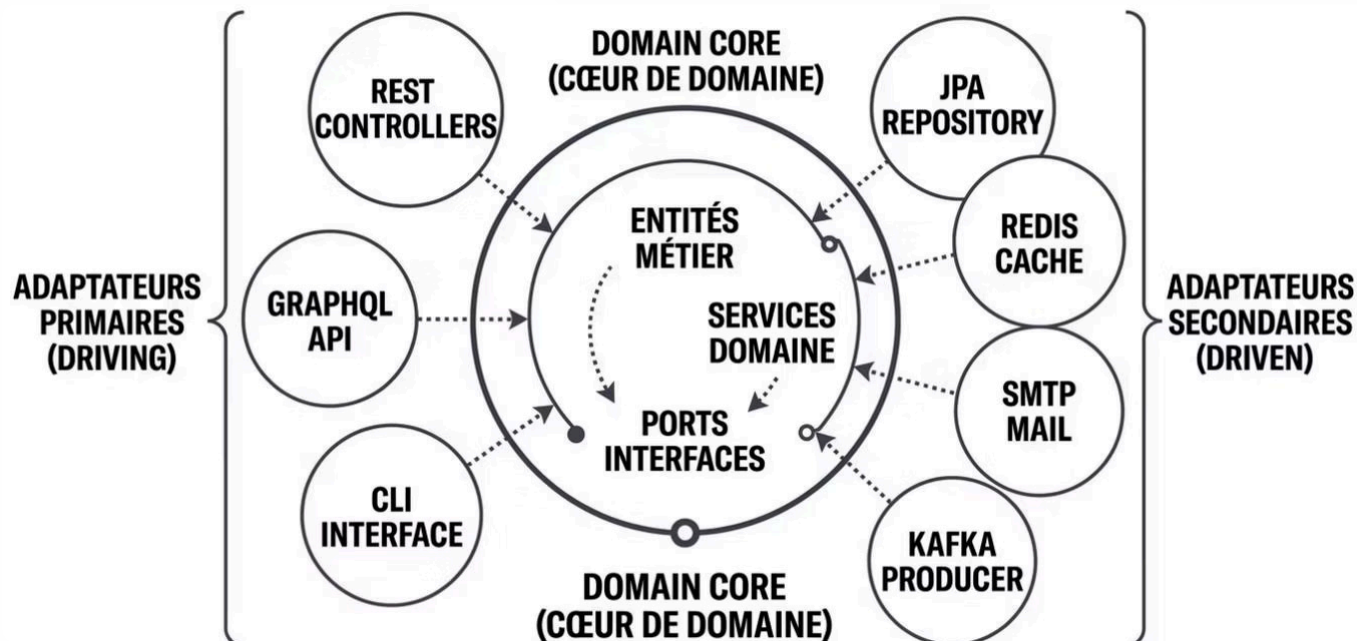
- Actuator health exposé
- Métriques Prometheus
- Logs structurés JSON



Résilience

- Timeouts sur appels externes
- Circuit breaker Resilience4j
- Graceful shutdown configuré

Architecture Hexagonale avec Spring



Ports & Adapters

Le domaine métier est isolé de tout framework. L'injection de dépendances Spring est le mécanisme clé pour inverser les dépendances.

- **Adaptateurs primaires** : REST, GraphQL, CLI
- **Cœur domaine** : entités, services, ports (interfaces)
- **Adaptateurs secondaires** : JPA, Redis, Kafka

i Pleinement justifiée pour les applications complexes à longue durée de vie — peut être surdimensionnée pour du CRUD simple.

CQRS avec Spring

Côté Commande (Write)

- Entités JPA riches avec logique domaine
- Validation stricte des invariants
- Publication d'événements domaine
- Transactions ACID
- Optimisé pour la **cohérence**

Côté Requête (Read)

- Projections SQL natives ou JPQL
- DTOs directement mappés depuis la BDD
- Pas d'entités JPA, lecture directe
- Cache agressif possible
- Optimisé pour la **performance**

Dans sa forme la plus simple : les commandes modifient l'état et retournent `void` ou un ID, les requêtes retournent des DTOs sans effet de bord.

Spring Data JPA – Aperçu

Spring Data JPA élimine la majorité du code répétitif d'accès aux données via des repositories génériques et la génération automatique de requêtes.

```
public interface UserRepository
    extends JpaRepository<User, Long> {

    Optional<User> findByEmail(String email);

    @Query("SELECT u FROM User u WHERE u.createdAt > :date")
    List<User> findUsersRegisteredAfter(
        @Param("date") LocalDateTime date);

    Page<User> findByActive(boolean active, Pageable pageable);

    List<UserSummary> findAllProjectedBy(); // Projection
}
```

 Toutes les méthodes d'écriture sont transactionnelles par défaut. Les méthodes de lecture utilisent `readOnly = true`.

Concepts JPA à maîtriser



Entités & mapping ORM

@Entity, @Table, @Column, stratégies
@GeneratedValue



Relations entre entités

@OneToMany, @ManyToOne, EAGER vs LAZY loading,
problème N+1



Cycle de vie des entités

New, Managed, Detached, Removed — crucial pour
éviter les LazyInitializationException



JPQL & Criteria API

Requêtes orientées objet avec JPQL, requêtes
dynamiques typées avec Criteria API